DAILY WORK PROGRESS REPORT

Unified Agentic AI Intelligence Platform

Date	13 October 2026	**Developer**	Aashish Puri
Module	RAG Chatbot	**Environment**	Python · VS Code · PowerShell

1. Objective

Begin development of the Retrieval-Augmented Generation (RAG) chatbot module and configure a language model through the Hugging Face Inference API.

2. Work Completed

- Reviewed the project requirements and selected the RAG chatbot as the starting module.
- Created the initial Python chatbot script at `rag-chatbot/main.py`.
- Integrated Hugging Face `InferenceClient` to send prompts to a language model.
- Configured the application to read the API token from the `HF_TOKEN` environment variable rather than hardcoding it in the source code.
- Ran the Python script and confirmed that the chatbot starts and accepts user input.
- Tested the application with a greeting and identified a model-availability issue.

3. Technologies Used

- Python and the `huggingface_hub` library
- Visual Studio Code
- Hugging Face Inference API
- PowerShell environment variables

4. Challenges and Troubleshooting

The first selected model, `HuggingFaceH4/zephyr-7b-beta`, was not supported by the Inference Providers enabled for the account. After changing the model identifier, the API returned **402 Payment Required**, stating that no Inference Provider credits remained. Therefore, the chatbot starts correctly, but response generation is blocked by provider credits/account availability.

5. Current Status

- **Initial chatbot setup:** Completed
- **Token environment variable loading:** Verified in the current terminal session
- **Successful AI response generation:** Pending due to provider credits
- **PDF extraction, chunking, embeddings, and retrieval:** Not yet implemented

The current application is a basic LLM chatbot, not yet a complete RAG system. RAG functionality will require document extraction, text chunking, embeddings, vector search, and answers grounded in retrieved passages with source references.

6. Plan for the Next Working Session

- Choose an available model/provider and resolve the credit or billing limitation.
- Test successful response generation.
- Implement PDF text extraction.
- Split extracted text into manageable chunks.
- Add embeddings and vector-based retrieval.
- Generate document-grounded answers with source references.

7. Conclusion

Today's work established the initial chatbot structure and verified that the application runs and accepts user input. The immediate priority is to resolve the Inference API credit limitation before implementing the core RAG pipeline.

SCREENSHOTS OF TODAY'S WORK

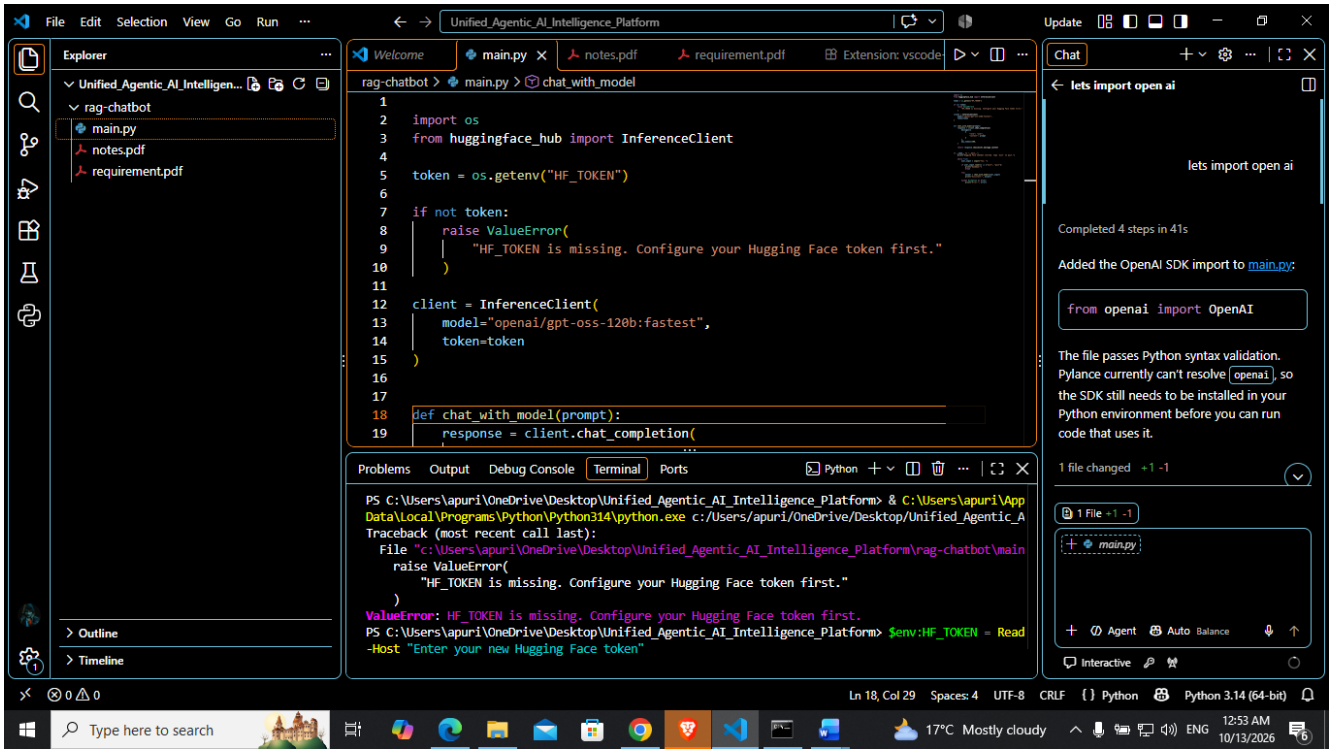


Figure 1. Chatbot configuration and Python code in VS Code.

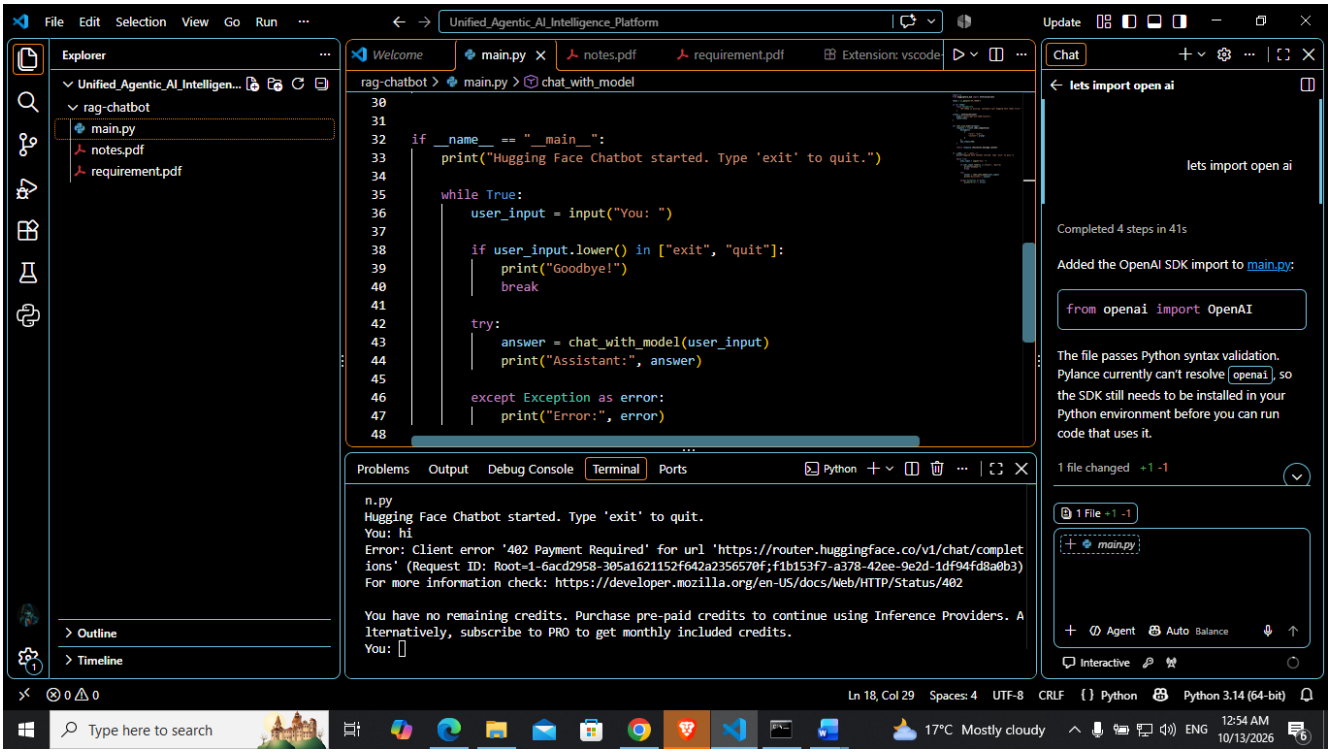


Figure 2. Chatbot test showing the Hugging Face Inference API credit error.